

深度学习与神经网络基础

一、前馈神经网络的详细数学

深度学习与神经网络基础

一、前馈神经网络的详细数学

1.1 通用近似定理

定理 (Cybenko, 1989; Hornik, 1991) :

设 σ 为非常值连续有界单调递增函数, 则对任意 $\epsilon > 0$ 、任意上的连续函数 f , 存在整数 m 、实常数 α_i, b_i 和向量 $w_i \in \mathbb{R}^d$

$$\left| f(x) - \sum_{i=1}^m \alpha_i \sigma(w_i^T x + b_i) \right| < \epsilon$$

直觉理解:

- 一个单隐藏层的神经网络可以近似任意连续函数到任意精度
- 但这并不意味着单隐藏层就是最优的: 定理只说 "存在", 没说需要多少神经元——可能需要指数级的神经元
- 深层网络可以用更少的参数表达同样的函数, 这就是 "深度" 的意义

类比: 就像多项式逼近——任何连续函数都可以用多项式逼近 (Weierstrass 定理), 但高次多项式远多于用分段函数的组合。

1.2 前向传播的矩阵形式推导

单层网络: 输入 $x \in \mathbb{R}^{d_{in}}$, 隐藏层 $h \in \mathbb{R}^{d_h}$, 输出层 $y \in \mathbb{R}^{d_{out}}$

符号定义:

- $W^{(1)} \in \mathbb{R}^{d_{in} \times d_h}$: 输入到隐藏层的权重
- $b^{(1)} \in \mathbb{R}^{d_h}$: 隐藏层偏置
- $W^{(2)} \in \mathbb{R}^{d_h \times d_{out}}$: 隐藏层到输出层的权重
- $b^{(2)} \in \mathbb{R}^{d_{out}}$: 输出层偏置
- σ : 激活函数

前向传播步骤:

$$z^{(1)} = x W^{(1)} + b^{(1)} \quad \text{(线性变换)}$$

$$h = \sigma(z^{(1)}) \quad \text{(非线性激活)}$$

$$z^{(2)} = h W^{(2)} + b^{(2)} \quad \text{(输出层线性变换)}$$

$$\hat{y} = \text{softmax}(z^{(2)}) \quad \text{(输出层非线性激活)}$$

批量前向传播（ n 个样本同时计算）：

设 $X \in \mathbb{R}^{n \times d_{in}}$ ，则：

$$Z^{(1)} = X W^{(1)} + b^{(1)} \quad (n \times d_{in} \times d_{h1})$$

$$H = \sigma(Z^{(1)}) \quad (n \times d_{h1})$$

$$Z^{(2)} = H W^{(2)} + b^{(2)} \quad (n \times d_{h1} \times d_{out})$$

```
import numpy as np
```

```
def forwardpass(X, params, activation='relu'):
    """
```

多层前馈神经网络的前向传播

params: 包含各层 W , b 的字典

```
"""
```

```
cache = {'A0': X} # 缓存用于反向传播
```

```
A = X
```

```
numlayers = len(params) // 2 # 每层有 W 和 b
```

```
for l in range(1, numlayers + 1):
```

```
    Z = A @ params[f'W{l}'] + params[f'b{l}']
```

```
    cache[f'Z{l}'] = Z
```

```
    if l == numlayers:
```

```
        # 最后一层不用激活函数（或用softmax）
```

```
        A = Z
```

```
    else:
```

```
        if activation == 'relu':
```

```
            A = np.maximum(0, Z)
```

```
        elif activation == 'sigmoid':
```

```

A = 1 / (1 + np.exp(-np.clip(Z, -500, 500)))
elif activation == 'tanh':
A = np.tanh(Z)

cache[f'A{l}'] = A

return A, cache

```

1.3 反向传播的完整数学推导

目标：计算损失函数对每个参数的梯度 $\frac{\partial L}{\partial b^{(l)}}$ 。

以三层网络（输入层 → 隐藏层 → 输出层）为例，使用均方误差损失：

$$L = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

推导步骤：

第一步：计算输出层梯度

$$\frac{\partial L}{\partial Z^{(2)}} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

记为 $\delta^{(2)}$

$$\frac{\partial L}{\partial W^{(2)}} = (A^{(1)})^T \delta^{(2)}$$

$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}$$

第二步：反向传播到隐藏层

$$\delta^{(1)} = \delta^{(2)} (W^{(2)})^T$$

其中 $\odot$ 是逐元素乘法（Hadamard 积）， σ' 是激活函数的导数。

$$\frac{\partial L}{\partial W^{(1)}} = X^T \delta^{(1)}$$

$$\frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}$$

$$\text{ReLU 的导数: } \sigma'(z) = \begin{cases} 1 & z > 0 \\ 0 & \text{otherwise} \end{cases}$$

一般化到 L 层网络：

$$\frac{\partial L}{\partial Z^{(L)}} = \frac{\partial L}{\partial Z^{(L)}}$$

$$\frac{\partial L}{\partial W^{(l)}} = \left(\frac{\partial L}{\partial Z^{(l+1)}} \right) W^{(l+1)} \quad \text{对 } l = L-1, \dots, 1$$

$$\frac{\partial L}{\partial b^{(l)}} = \text{mean of } \frac{\partial L}{\partial Z^{(l)}}$$

```
def backwardpass(cache, params, Y, activation='relu'):
```

多层前馈神经网络的反向传播

```
"""
```

```
    grads = {}
```

```
    numlayers = len(params) // 2
```

```
    n = Y.shape[0]
```

```
    # 输出层梯度 (MSE 损失)
```

```
    A_final = cache[f'A{numlayers}']
```

```
    dA = (A_final - Y) / n # dL/dA
```

```
    for l in reversed(range(1, numlayers + 1)):
```

```
        A_prev = cache[f'A{l-1}']
```

```
        Z = cache[f'Z{l}']
```

```
    # 计算 dL/dZ
```

```
    if l == numlayers:
```

```
        # 输出层: dL/dZ = dL/dA (无激活或 softmax)
```

```
        dZ = dA
```

```
    else:
```

```
        # 隐藏层: dZ = dA * σ'(Z)
```

```
        if activation == 'relu':
```

```
            dZ = dA * (Z > 0).astype(float)
```

```
        elif activation == 'sigmoid':
```

```
            s = 1 / (1 + np.exp(-np.clip(Z, -500, 500)))
```

```
            dZ = dA * s * (1 - s)
```

```

elif activation == 'tanh':
    dZ = dA * (1 - np.tanh(Z) ** 2)

# 计算 dL / dW 和 dL / db
grads[f'W{l}'] = A_prev.T @ dZ
grads[f'b{l}'] = dZ.sum(axis=0)

# 传播到上一层
if l > 1:
    dA = dZ @ params[f'W{l}'].T

return grads

```

1.4 计算图的概念和自动微分

计算图：将数学运算表示为有向无环图（DAG），节点是变量或运算，边是数据流。

示例： $f(x, y, z) = (x + y) \cdot z$ ，设 $x = 2$ ， $y = -3$ ， $z = 4$

计算图：

```

x  →  [ + ]  →  [ × ]  →  f
y  →  [ + ]  ↑
z  ─┘

```

前向传播： $q = x + y = -1$ ， $f = q \times z = -4$

反向传播： $\partial f / \partial q = z = 4$ ， $\partial f / \partial z = q = -1$

$\partial f / \partial x = \partial f / \partial q \times \partial q / \partial x = 4 \times 1 = 4$

$\partial f / \partial y = \partial f / \partial q \times \partial q / \partial y = 4 \times 1 = 4$

自动微分 vs 数值微分 vs 符号微分：

方法	原理	精度	效率
符号微分	操作表达式树	精确	可能表达式膨胀
数值微分	$\frac{f(x + \epsilon) - f(x)}{\epsilon}$	近似（截断误差和舍入误差）	低
自动微分	计算图上的链式法则	精确	$O(1)$ 次前向传播

PyTorch 的自动微分：

```

import torch

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(-3.0, requires_grad=True)
z = torch.tensor(4.0, requires_grad=True)

f = (x + y) * z
f.backward()

print(f"f = {f.item()}") # -4
print(f"df/dx = {x.grad.item()}") # 4
print(f"df/dy = {y.grad.item()}") # 4
print(f"df/dz = {z.grad.item()}") # -1

- - -

```

二、激活函数的详细分析

2.1 每个激活函数的导数特性

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- 最大导数值： $\sigma'(0) = 0.25$ （在 $x=0$ 处）
- 当 $|x|$ 较大时， $\sigma'(x) \approx 0$ （饱和区）
- 致命缺陷：最大导数仅为 0.25 ，经过多层后梯度按 0.25^L 指数衰减

Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad \tanh'(x) = 1 - \tanh^2(x)$$

- 最大导数值： $\tanh'(0) = 1$
- 比 Sigmoid 好：零均值、导数更大
- 但在饱和区仍然趋近于零

ReLU

$$\text{ReLU}(x) = \max(0, x), \quad \text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

- 正区间导数恒为 1，彻底解决正区间的梯度消失
- 计算极快（只需比较和 $\max$ 操作）
- 0 点处不可微，但在实际中不影响

Leaky ReLU / PReLU

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

- 解决 Dead ReLU 问题：负区间仍有非零梯度
- PReLU 将 α 作为可学习参数

GELU

$$\text{GELU}(x) = x \cdot \Phi(x) = x \cdot \frac{1}{2} \left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right]$$

$$\text{近似: } \text{GELU}(x) \approx 0.5x \left(1 + \tanh\left[\frac{x}{\sqrt{2}}\right] \right)$$

- Transformer 默认激活函数
- 在 $x = 0$ 处有平滑的软门控效果
- 结合了 ReLU 的稀疏性和 sigmoid 的平滑性

Swish

$$\text{Swish}(x) = x \cdot \sigma(\beta x)$$

- 当 $\beta \rightarrow \infty$ 时退化为 ReLU
- 实验表明在深层网络中优于 ReLU

2.2 梯度消失 / 爆炸问题的数学解释

问题根源：

考虑 L 层网络，每层使用 Sigmoid 激活。反向传播时，梯度通过链式法则连乘：

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(L)} \prod_{l=2}^L \sigma'(W^{(l-1)} a^{(l-1)})$$

$$\left[\sigma'(z^{(l)}) \right]$$

对于 Sigmoid: $\sigma'(z) \leq 0.25$ ，如果权重 $W^{(l)}$

经过 L 层后，梯度被缩放 $< 0.25^L$ ：

- $L = 10$ ： $< 10^{-6}$
- $L = 20$ ： $< 10^{-12}$

梯度爆炸同理：当 $W \cdot \sigma' > 1$ 时，梯度指数增长。

解决方案对比：

方案 原理 适用场景

ReLU 正区间导数恒为 1 通用

残差连接 梯度通过跳跃连接直接回传 深层网络

BatchNorm 标准化每层输出，防止进入饱和区 CNN

梯度裁剪 限制梯度范数 RNN

合理初始化 使各层梯度方差一致 所有权重初始化

LSTM门控 选择性记忆，控制梯度流 序列建模

2.3 Dead ReLU 问题

原因：当 $w x + b < 0$ 时，ReLU 输出为 0，梯度也为 0。如果某个神经元对所有输入都为 0，它的权重将永远不会更新——这个神经元“死了”。

触发条件：

- 学习率过大，权重更新幅度过大
- 初始化偏置为较大负值
- 数据分布变化

检测方法：统计训练过程中各层 ReLU 输出为 0 的比例。如果超过 50%，说明大量神经元死亡。

解决方案：

1. 使用 Leaky ReLU：负区间保留小梯度
2. 降低学习率
3. 偏置初始化为小正值（如 0.01）
4. 使用带参数的 PReLU

2.4 为什么需要非线性激活函数

反证法：如果没有非线性激活函数，多层网络退化为单层线性变换。

证明：对于两层线性网络：

$$\begin{aligned} h &= W^{(1)} x + b^{(1)} \\ \hat{y} &= W^{(2)} h + b^{(2)} = W^{(2)} (W^{(1)} x + \underbrace{b^{(1)}}_{b'}) + \underbrace{b^{(2)}}_{b''} \end{aligned}$$

结果仍为线性变换 $W' x + b'$ ，增加层数毫无意义。

非线性激活函数使网络能够学习非线性决策边界，这是深度学习表达能力的基础。

- - -

三、损失函数的详细讨论

3.1 交叉熵损失的推导和信息论解释

从最大似然估计推导

假设模型输出类别概率 $P(y|x; \theta)$ ，样本独立同分布，似然函数为：

$$L(\theta) = \prod_{i=1}^n P(y_i|x_i; \theta)$$

对数似然：

$$\ell(\theta) = \sum_{i=1}^n \log P(y_i|x_i; \theta)$$

取负号得到交叉熵损失：

$$\mathcal{L}_{CE} = - \sum_{i=1}^n \log P(y_i|x_i; \theta)$$

对于二分类：

$$\mathcal{L}_{CE} = - \frac{1}{n} \sum_{i=1}^n [\log(\hat{p}_i) + (1 - \hat{p}_i) \log(1 - \hat{p}_i)]$$

对于多分类：

$$\mathcal{L}_{CE} = -\frac{1}{n} \sum_{i=1}^n \sum_k y_{ik} \log \hat{p}_{ik}$$

其中 y_{ik} 是 one-hot 编码, $\hat{p}_{ik} = \text{softmax}(\mathbf{z}_{ik})$

信息论解释

信息熵（不确定性度量）：

$$H(Y) = -\sum_{k=1}^K P(y=k) \log P(y=k)$$

交叉熵（用分布 Q 编码分布 P 所需的平均比特数）：

$$H(P, Q) = -\sum_k P(k) \log Q(k)$$

KL 散度（ Q 近似 P 的额外代价）：

$$D_{KL}(P \parallel Q) = H(P, Q) - H(P) = \sum_k P(k) \log \frac{P(k)}{Q(k)}$$

关键关系：

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

因为 $H(P)$ 是常数（由数据分布决定），最小化交叉熵等价于最小化 KL 散度，即让模型分布尽可能接近真实分布 P 。

3.2 MSE 的优缺点

均方误差：

$$\mathcal{L}_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

优点：

- 凸函数，有唯一最小值
- 可微，梯度简单
- 对大误差惩罚更重

缺点：

- 对异常值敏感
- 在分类问题中，与 Sigmoid / Tanh 搭配时梯度可能消失（因为 $\sigma'(z)$ 很小）

- 不适合概率输出（不满足概率公理）

MSE + Sigmoid 的梯度消失问题：

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} =$$

当 $\hat{y}$ 接近 0 或 1 时（饱和区）， $\sigma'(z) \approx 0$ ，梯度也接近零。

而交叉熵 + Sigmoid 的梯度：

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z} =$$

梯度只与误差有关，不受 σ' 影响——这是分类问题首选交叉熵的原因。

3.3 Focal Loss 的动机和公式

问题：在目标检测等任务中，正负样本极度不平衡（如 1 : 1000），且大部分负样本很容易分类（easy negatives）。标准交叉熵对这些简单样本的损失虽然小，但累积起来会淹没稀有类别的梯度信号。

Focal Loss：

$$\mathcal{L}_{\text{FL}} = -\alpha^t (1 - p)^{\gamma} \log p$$

其中：

- $p^t = \begin{cases} p & y=1 \\ 1-p & y=0 \end{cases}$
- $\gamma \geq 0$ 是聚焦参数
- α 是类别平衡权重

$(1 - p)^{\gamma}$ 的作用：

- 当样本容易分类（ $p \rightarrow 1$ ）时， $(1 - p)^{\gamma} \rightarrow 0$ ，损失被抑制
- 当样本难分类（ $p \rightarrow 0$ ）时， $(1 - p)^{\gamma} \rightarrow 1$ ，损失保持不变
- γ 越大，对简单样本的抑制越强

数值示例：

- 简单样本 $p = 0.9$ ：CE 损失 = 0.105；FL 损失（ $\gamma = 2$ ） = 0.00105，降低 100 倍
- 困难样本 $p = 0.3$ ：CE 损失 = 1.204；FL 损失（ $\gamma = 2$ ） = 0.590，降低约 2 倍

```

import torch
import torch.nn.functional as F

def focalloss(pred, target, alpha=0.25, gamma=2.0):
    """Focal Loss实现"""
    bce = F.binarycrossentropywithlogits(pred, target)
    pt = torch.exp(-bce) # pt = exp(-CE)
    focalweight = (alpha * target + (1 - alpha) * (1 - target)) ** gamma
    return (focalweight * bce).mean()

- - -

```

四、优化算法与训练技巧

4.1 梯度下降变体对比

算法类型 更新规则 优点 缺点

批量梯度下降 $\theta = \theta - \eta \nabla_{\theta} J(\theta)$

随机梯度下降 $\theta = \theta - \eta \nabla_{\theta} J(\theta)$
方差大，不稳定

小批量梯度下降 $\theta = \theta - \eta \nabla_{\theta} J(\theta)$
需调批量大小

批量大小的影响：

- 小批量（16 - 32）：正则化效果，泛化更好，但训练慢
- 大批量（256 - 1024）：训练快，但可能收敛到尖锐极小值，泛化差
- 经验法则：大批量需要成比例增加学习率（线性缩放规则：批量翻倍，学习率翻倍）

4.2 自适应优化算法详解

Adam 的完整推导

初始化： $m_0 = 0$, $v_0 = 0$, $t = 0$

每步更新：

$t = t + 1$

```


$$g_t = \nabla_{\theta} J(\theta_{t-1}) \quad \text{(计算当前梯度)}$$


$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad \text{(计算带衰减的梯度)}$$


$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{(计算平方梯度的带衰减平均值)}$$


$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{(偏置修正后的梯度估计)}$$


$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(偏置修正后的平方梯度估计)}$$


$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t}} \hat{m}_t \quad \text{(参数更新)}$$


```

为什么需要偏差修正？

初始 $m_0 = 0$ ，因此早期 m_t 偏小： $m_1 = (1 - \beta_1) g_1$ 。除以 $1 - \beta_1^t$ 可以放大早期估计，修正初始化偏差。当 $t \rightarrow \infty$ 时， $\beta_1^t \rightarrow 0$ ，修正因子趋近于1。

默认超参数： $\beta_1 = 0.9$ ， $\beta_2 = 0.999$ ， $\epsilon = 10^{-9}$

Adam优化器完整实现

```

class Adam:
    def __init__(self, params, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-9):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = epsilon
        self.t = 0
        self.m = [np.zeros_like(p) for p in params]
        self.v = [np.zeros_like(p) for p in params]

    def step(self, params, grads):
        self.t += 1
        for i, (p, g) in enumerate(zip(params, grads)):
            self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * g
            self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) * g**2

            m_hat = self.m[i] / (1 - self.beta1**self.t)
            v_hat = self.v[i] / (1 - self.beta2**self.t)

            params[i] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

```

4.3 学习率调度策略

策略 公式 / 说明 适用场景

预热策略 初始阶段线性增加学习率 大批量训练、Transformer

余弦退火 $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \cos\left(\frac{\pi t}{T}\right)$ 通用

Step 衰减 每 N 个 epoch 衰减一次 经典 CNN

自适应调整 验证指标不变时降低学习率 不确定最优调度时

Warm Restart 余弦退火 + 周期性重启 逃离局部最优

...

五、正则化与泛化技术

5.1 经典正则化方法

方法 说明 原理

L2 正则化 $J_{\text{new}} = J + \frac{\lambda}{2n} \sum w^2$ 权重衰减

L1 正则化 $J_{\text{new}} = J + \frac{\lambda}{n} \sum |w|$ 权重稀疏化

Dropout 训练时随机屏蔽部分神经元 隐式模型集成

早停法 验证集性能不再提升时停止 限制有效训练轮数

数据增强 扩充训练数据 增加数据多样性

Dropout 的数学原理：

训练时：以概率 p 将神经元置零，其余神经元输出乘以 $\frac{1}{1-p}$ (inverse dropout，推理时无需缩放)

$\tilde{h} = m \odot h$, $m_i \sim \text{Bern}(p)$

推理时：使用所有神经元，不进行随机屏蔽。

Dropout $\approx$ 模型集成：每次 Dropout 采样相当于训练一个子网络，推理时使用所有子网络

Dropout 实现示例

```
def dropout_layer(x, p_dropout, training=True):  
    if not training or p_dropout <= 0:
```

```

return x
mask = (np.random.rand(x.shape) > p_dropout) / (1
return x * mask

```

5.2 高级正则化技术

标签平滑：

将硬标签 $[0, 1]$ 变为软标签 $[\epsilon/K, 1 - \epsilon + \epsilon/K]$

$$y_{smooth} = (1 - \epsilon) y_{hard} + \epsilon / K$$

作用：防止模型过度自信，提高校准性。

Mixup：

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha)$$

梯度裁剪：

梯度裁剪（防止梯度爆炸）

```

torch.nn.utils.clip_grad_norm(model.parameters(), max_norm)

- - -

```

六、卷积神经网络（CNN）

6.1 卷积操作的数学定义和直觉

数学定义：

二维离散卷积：

$$(f * g)[i, j] = \sum_m \sum_n f[m, n] \cdot g[i - m, j - n]$$

在深度学习中实际使用的是互相关（cross-correlation）（不翻转卷积核）：

$$(f \star g)[i, j] = \sum_m \sum_n f[m, n] \cdot g[i - m, j - n]$$

输出尺寸计算：

$$s_o = \lfloor \frac{i - k + 2p}{s} \rfloor + 1$$

其中 i = 输入尺寸, k = 卷积核尺寸, p = padding, s = stride

三大设计原则：

1. 局部连接：每个神经元只与输入的局部区域连接，而非全连接

- 依据：图像的局部相关性（相邻像素高度相关）
- 参数量：从 $d_{in} \times d_{out}$ 降到 $k^2 \times d_{out}$

2. 参数共享：同一卷积核在所有位置使用相同权重

- 依据：平移不变性（同一个特征可能出现在任何位置）
- 参数量进一步降低 $H \times W$ 倍

3. 层次化特征：低层学习边缘 / 纹理，高层学习语义 / 对象

```
import torch
import torch.nn as nn
```

卷积操作详解

```
conv = nn.Conv2d(
    in_channels=3, # 输入通道数 (RGB)
    out_channels=64, # 输出通道数 (卷积核数量)
    kernel_size=3, # 卷积核大小
    stride=1, # 步长
    padding=1, # 填充 (保持尺寸: padding = (k - 1) / 2)
    bias=True
)
```

参数量计算

```
params = 3 * 3 * 3 * 64 + 64 # (inch * k^2 * outch) + bias
print(f"卷积层参数量: {params}")
```

6.2 感受野的计算

感受野：输出特征图上某个像素对应的输入图像上的区域大小。

递推公式：

$$r_l = r_{l-1} + (k_l - 1) \times \prod_{i=1}^{l-1} s_i$$

其中 r_l 是第 l 层的感受野， k_l 是第 l 层的卷积核大小， s_i 是第 i 层的步长。

示例：3层 3×3 卷积 ($stride = 1$) 的感受野

- $r_1 = 1 + (3 - 1) \times 1 = 3$
- $r_2 = 3 + (3 - 1) \times 1 = 5$
- $r_3 = 5 + (3 - 1) \times 1 = 7$

关键结论：2层 3×3 卷积的感受野等于1层 5×5 卷积，但参数更少 ($2 \times 3^2 = 18$ vs $1 \times 5^2 = 25$)，且多了一层非线性。这就是VGG使用小卷积核的原理。

6.3 经典架构详细说明

LeNet-5 (1998)

Input ($32 \times 32 \times 1$) $\rightarrow$ Conv (5×5 , 6) $\rightarrow$ Pool (2×2) $\rightarrow$ Conv (5×5 , 16) $\rightarrow$ Pool (2×2) $\rightarrow$ FC (10)

- 首次将CNN应用于手写数字识别
- 使用 Sigmoid / Tanh 激活
- 约6万参数

AlexNet (2012)

Input ($224 \times 224 \times 3$) $\rightarrow$ Conv (11×11 , 96, $s=4$) $\rightarrow$ Conv (5×5 , 128, $s=2$) $\rightarrow$ Conv (3×3 , 128, $s=2$) $\rightarrow$ Conv (3×3 , 256, $s=2$) $\rightarrow$ Conv (3×3 , 256, $s=2$) $\rightarrow$ Conv (3×3 , 512, $s=2$) $\rightarrow$ Conv (3×3 , 512, $s=2$) $\rightarrow$ Conv (3×3 , 512, $s=2$) $\rightarrow$ FC (4096) $\rightarrow$ FC (1000)

关键创新：

- ReLU 激活 (替代 Sigmoid, 解决梯度消失)
- Dropout (FC层, $p = 0.5$)
- 数据增强 (随机裁剪、水平翻转、颜色抖动)
- GPU 并行训练
- 局部响应归一化 (LRN, 后来被 BatchNorm 替代)

VGGNet (2014)

核心设计规则：

- 只用 3×3 卷积和 2×2 最大池化
- 每次池化后通道数翻倍
- VGG-16: 13 层卷积 + 3 层全连接 ≈ 1.38 亿参数

贡献：证明了 " 更深的网络 + 更小的卷积核 " 是有效策略。

GoogLeNet / Inception (2014)

Inception 模块：并行使用不同大小的卷积核 (1×1 , 3×3 , 5×5) + 最大池化，然后拼

1×1 卷积的作用：

1. 降维：减少通道数，降低计算量 (" 瓶颈 " 层)
2. 跨通道信息融合：等价于在通道维度上的全连接
3. 增加非线性：后接 ReLU，增加表达能力

示例：将 256 通道降到 64 通道，再升回 256 通道

- 直接 3×3 卷积: $256 \times 3 \times 3 \times 256 \approx 6.8$ 亿参数
- 瓶颈结构 ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$): $256 \times 1 \times 1 \times 64 \times 1 \times 1 \times 256 \approx 6.8$ 万参数

参数减少近 9 倍！

ResNet (2015)

残差学习的核心思想：

不直接学习映射 $H(x)$ ，而是学习残差 $F(x) = H(x) - x$ ，即：

$$H(x) = F(x) + x$$

为什么残差学习更有效？

1. 梯度直通：反向传播时，梯度可以通过跳跃连接直接回传： $\frac{\partial H}{\partial x} = \frac{\partial F}{\partial x} + 1$ ，至少为 1
2. 恒等映射容易学习：如果最优映射就是恒等映射，只需让 $F(x) \rightarrow 0$ ，比直接学习 $H(x)$

更容易

3. 信息高速公路：信息可以绕过任意层直接传递

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, stride)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(channels)
        self.relu = nn.ReLU(inplace=True)

        self.shortcut = nn.Sequential()
        if stride != 1:
            self.shortcut = nn.Sequential(
                nn.Conv2d(channels, channels, 1, stride=stride, padding=0),
                nn.BatchNorm2d(channels)
            )

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += identity # 残差连接
        out = self.relu(out)
        return out
```

DenseNet (2017)

密集连接：每层接收前面所有层的输出作为输入：

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

优点：特征复用、梯度传播更好、参数更少。缺点：内存消耗大（特征图拼接）。

EfficientNet (2019)

复合缩放：同时缩放深度、宽度和分辨率：

$$\text{depth} : \alpha^\phi, \quad \text{width} : \gamma^\phi$$

约束 $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$
 ≥ 1 。

6.4 空洞卷积和可变形卷积

空洞卷积 (Dilated / Atrous Convolution)：

在卷积核元素之间插入空洞 (holes)，扩大感受野而不增加参数。

- 空洞率 r ：卷积核元素间距为 $r - 1$
- 有效卷积核大小： $k_{\text{eff}} = k + (k - 1)(r - 1) = k \cdot r$
- 例： 3×3 卷积， $r = 2$ $\rightarrow$ 有效大小 5×5 ，参数仍为 9 个

应用：语义分割中需要大感受野但不能降低分辨率（不能用池化）

可变形卷积 (Deformable Convolution)：

为每个卷积核位置学习一个偏移量 $(\Delta m, \Delta n)$ ，使卷积核可以变形以适

- - -

七、循环神经网络 (RNN)

7.1 基础 RNN

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b_h)$$
$$y_t = W_{hy} h_t + b_y$$

7.2 BPTT (沿时间反向传播) 的推导

展开 RNN：将 RNN 按时间步展开，形成一个深层前馈网络。

损失函数：

$$L = \sum_{t=1}^T L_t$$

对 W_{hh} 的梯度：

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T$$

以第 t 步为例：

$$\frac{\partial L_t}{\partial W_{hh}} = \sum_{k=1}^T \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}}$$

关键项 $\frac{\partial h_t}{\partial h_k}$ 的链式展开：

$$\frac{\partial h_t}{\partial h_k} = \prod_{i=k+1}^t \text{diag}(\sigma'(z_i)) W_{hh}$$

梯度消失 / 爆炸分析：

当 T 很大时，连乘项 $\prod W_{hh} \cdot \text{diag}(\sigma'(z_i))$

- 最大特征值 > 1 $\rightarrow$ 梯度爆炸
- 最大特征值 < 1 $\rightarrow$ 梯度消失

截断 BPTT：实践中不展开全部时间步，而是截断到 k 步（如 $k=20$ ），只考虑最近 k

7.3 LSTM 如何解决梯度消失

LSTM 的核心：引入细胞状态 C_t 作为 "信息高速公路"，通过门控机制控制信息流动。

公式：

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{遗忘门} \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{输入门} \\ \tilde{C}_t &= \tanh(W_C[h_{t-1}, x_t] + b_C) \quad \text{候选细胞状态} \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{细胞状态更新} \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{输出门} \\ h_t &= o_t \odot \tanh(C_t) \quad \text{输出隐藏状态} \end{aligned}$$

LSTM 的梯度路径：

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t$$

如果 $f_t \approx 1$ ，梯度几乎无损传递；如果 $f_t \approx 0$ ，旧信息被

与普通RNN的关键区别：

- RNN： $\frac{\partial h_t}{\partial h_{t-1}} = \text{tanh}(h_{t-1})$ 的连乘

- LSTM： $\frac{\partial C_t}{\partial C_{t-1}} = f_t$

遗忘门 f_t 是一个可学习的标量（0到1之间），网络可以学会让重要的长期信息不被遗忘。

7.4 GRU 的简化设计

GRU (Gated Recurrent Unit)：将遗忘门和输入门合并为更新门：

$$z_t = \sigma(W_z [h_{t-1}, x_t]) \quad \text{更新门}$$

$$r_t = \sigma(W_r [h_{t-1}, x_t]) \quad \text{重置门}$$

$$\tilde{h}_t = \tanh(W [r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

LSTM vs GRU：

维度 LSTM GRU

门控数量 3个（遗忘、输入、输出） 2个（更新、重置）

参数量 较多 较少（约少1/3）

细胞状态 有独立的 C_t 无，直接用 h_t

表达能力 更强 足够，训练更快

适用场景 复杂长序列 数据量较少、序列较短

```
import torch
```

```
import torch.nn as nn
```

```
class LSTMModel(nn.Module):
```

```
def __init__(self, inputsize, hidsize, numlayers=2):
```

```

super().init()
self.lstm = nn.LSTM(
    inputsize=inputsize,
    hidden_size=hidden_size,
    numlayers=numlayers,
    batchfirst=True,
    dropout=dropout,
    bidirectional=True # 双向LSTM
)
self.fc = nn.Linear(hidden_size * 2, numclasses) # >

def forward(self, x):
# x: (batch, seqlen, inputsize)
output, (hn, cn) = self.lstm(x)
# 取最后一个时间步的输出（拼接前向和后向）
out = torch.cat([hn[-2], hn[-1]], dim=1) # 双向最后一层
return self.fc(out)

- - -

```

八、注意力机制与Transformer

8.1 自注意力计算

$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$

为什么要除以 $\sqrt{d_k}$ ？

当 d_k 较大时，点积 $q^T k$ 的方差为 d_k （假设各维度独立），可能导致 softmax 输入值过大，进入饱和区（梯度接近0）。除以 $\sqrt{d_k}$ 将方差归一化为1。

计算复杂度： $O(n^2 \cdot d)$ ，其中 n 是序列长度， d 是维度。这是Transformer的瓶颈。

8.2 多头注意力

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_i)$
 $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

每个头学习不同的注意力模式（如一个关注语法，一个关注语义）。

8.3 位置编码

由于自注意力是排列不变的（打乱输入顺序不影响输出），需要注入位置信息。

正弦位置编码：

$$\begin{aligned}PE(\text{pos}, 2i) &= \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right) \\PE(\text{pos}, 2i+1) &= \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)\end{aligned}$$

设计理由：对于任意固定偏移 k ， $PE(\text{pos} + k)$ 可以表示为 $PE(\text{pos})$ 的线性函数，使模型能够学习相对位置关系。

Transformer 详解见「重点能力 → Transformer 与大语言模型原理」

九、生成模型

9.1 GAN：训练动态和模式崩塌

框架：

生成器 G ：从噪声 z 生成假样本 $G(z)$

判别器 D ：区分真样本和假样本

目标函数：

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [D(x) - D(G(z))]$$

训练动态分析：

- 最优判别器： $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$
- 最优生成器： $p_g = p_{\text{data}}$ 时， $D^*(x) = 0.5$
- 训练过程： D 和 G 交替优化，类似双人零和博弈

模式崩塌 (Mode Collapse)：

生成器只生成少数几种样本，忽略了数据分布的多样性。

原因：生成器发现某种模式能 " 骗过 " 判别器后，不断生成该模式，失去多样性。

解决方案：

- WGAN：用Wasserstein距离替代JS散度，提供更有意义的梯度
- Spectral Normalization：对判别器权重做谱归一化，满足Lipschitz约束
- Unrolled GAN：生成器更新时考虑判别器的未来更新
- 多样性正则化：惩罚生成样本之间的相似度

GAN训练伪代码

```
for epoch in range(numepochs):
    # 训练判别器
    realdata = samplerrealdata(batchsize)
    z = samplenoise(batchsize)
    fakedata = G(z)

    dloss = -torch.mean(torch.log(D(realdata))) + torch.mean(torch.log(1 + D(fakedata)))
    doptimizer.step()

    # 训练生成器
    z = samplenoise(batchsize)
    fakedata = G(z)
    gloss = -torch.mean(torch.log(D(fakedata))) # 最大化
    goptimizer.step()
```

9.2 VAE：变分推断推导

目标：学习数据的生成模型 $p_{\theta}(x) = \int p_{\theta}(x|z) p(z) dz$

问题：积分不可解，后验 $p_{\theta}(z|x) = \frac{p_{\theta}(x|z) p(z)}{\int p_{\theta}(x|z) p(z) dz}$

变分推断：用 $q_{\phi}(z|x)$ 近似 $p_{\theta}(z|x)$ 。

ELBO 推导：

$$-\log p_{\theta}(x) = -\log \int p_{\theta}(x|z) p(z) dz$$

利用 Jensen 不等式：

$$\mathbb{E}_{q(\phi)}[\log p(\theta(x))] = \mathbb{E}_{q(\phi)}[\log p(\theta(x))] - \mathbb{E}_{q(\phi)}[D_{KL}(q(\phi) \parallel p(\theta(x)))]$$

因为 $D_{KL} \geq 0$ ，所以：

$$\mathbb{E}_{q(\phi)}[\log p(\theta(x))] \geq \mathbb{E}_{q(\phi)}[\log p(\theta(x))] - \mathbb{E}_{q(\phi)}[D_{KL}(q(\phi) \parallel p(\theta(x)))]$$

最大化 ELBO 同时：

1. 最大化重构质量 $\mathbb{E}_{q(\phi)}[\log p(\theta(x, z))]$
2. 最小化近似后验与先验的 KL 散度 $D_{KL}(q(\phi) \parallel p)$

重参数化技巧：

直接从 $q(\phi(z, x))$ 采样不可微。将采样改为确定性变换：

$$z = \mu(\phi(x)) + \sigma(\phi(x)) \odot \epsilon$$

梯度通过 μ, σ 传递，随机性由 ϵ 引入。

```
import torch
import torch.nn as nn

class VAE(nn.Module):
    def __init__(self, inputdim, hiddendim, latentdim):
        super().__init__()
        # 编码器
        self.encoder = nn.Sequential(
            nn.Linear(inputdim, hiddendim),
            nn.ReLU(),
        )
        self.fc_mu = nn.Linear(hiddendim, latentdim)
        self.fc_logvar = nn.Linear(hiddendim, latentdim)
        # 解码器
```

```

self.decoder = nn.Sequential(
    nn.Linear(latentdim, hiddendim),
    nn.ReLU(),
    nn.Linear(hiddendim, inputdim),
    nn.Sigmoid(),
)

def reparameterize(self, mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std

def forward(self, x):
    h = self.encoder(x)
    mu, logvar = self.fcmu(h), self.fclogvar(h)
    z = self.reparameterize(mu, logvar)
    xrecon = self.decoder(z)
    return xrecon, mu, logvar

def vaeloss(xrecon, x, mu, logvar):
    """ 重构损失 + KL 散度 """
    reconloss = nn.functional.binarycrossentropy(xrecon, x)
    # KL 散度解析解 (两个高斯分布之间)
    klloss = -0.5 * torch.sum(1 + logvar - mu.pow(2) -
                               torch.exp(logvar))
    return reconloss + klloss

```

9.3 扩散模型：前向加噪和反向去噪

前向过程（加噪）：

给定数据 x_0 ，逐步添加高斯噪声：

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

直接采样任意步：

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

$\mathcal{N}(0, 1)$

其中 $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$

当 $T \rightarrow \infty$ 时, x_T 趋近于标准高斯噪声。

反向过程（去噪）：

学习从 x_t 恢复 x_{t-1} ：

$p_{\theta}(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$

关键发现：最优 μ_{θ} 可以通过预测噪声 ϵ 来参数化：

$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$

训练目标（简化版）：

$\mathcal{L}_{\text{simple}} = \mathbb{E}_t [\|x_0 - \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t))\|^2]$

即：网络学习从带噪数据 x_t 中预测加入的噪声 ϵ 。

推理过程：从 $x_T \sim \mathcal{N}(0, 1)$ 开始，逐步去噪：

$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, 1)$

扩散模型简化训练流程

```
def train_step(model, x0, optimizer):
```

```
    batch_size = x0.shape[0]
```

```
    # 随机采样时间步
```

```
    t = torch.randint(0, T, (batch_size,)).to(device=x0.device)
```

```
    # 采样噪声
```

```
    noise = torch.randn_like(x0)
```

```
    # 前向加噪
```

```

x_t = sqrt(alphabar[t]).view(-1, 1, 1, 1) x_0 + \
sqrt(1 - alphabar[t]).view(-1, 1, 1, 1) noise

# 预测噪声
noise_pred = model(x_t, t)

# 计算损失
loss = nn.functional.mse_loss(noise_pred, noise)

optimizer.zero_grad()
loss.backward()
optimizer.step()

return loss.item()

```

9.4 生成模型对比

维度 GAN VAE 扩散模型

训练稳定性 差（模式崩塌） 好 好

生成质量 高 中等 最高

多样性 易缺失 好 好

似然估计 无 下界（ELBO） 可计算

采样速度 快（单步） 快（单步） 慢（多步迭代）

理论基础 博弈论 变分推断 随机微分方程

- - -

十、Batch Normalization 的详细原理

10.1 算法描述

训练时：

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \quad \text{}$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad \text{}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad \text{}$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{(缩放和平移)}$$

其中 γ 和 β 是可学习参数， ϵ 是数值稳定常数。

推理时：

使用训练时积累的全局均值和方差 (running mean/var)：

$$\hat{x} = \frac{x - \mu_{\text{running}}}{\sqrt{\sigma^2_{\text{running}} + \beta}}$$

10.2 为什么有效？

假说一：内部协变量偏移 (Internal Covariate Shift)

原始论文的观点：每层输入的分布在训练过程中不断变化，导致训练困难。BN 通过标准化解决了这个问题。

反驳：Santurkar et al. (2018) 通过实验表明，BN 的效果与减少内部协变量偏移无关。即使层输入分布变化不大，BN 仍然有效；反之，不使用 BN 但控制分布不变，训练仍然困难。

假说二：损失曲面平滑 (更可信)

BN 使损失函数的 Lipschitz 常数更小，梯度更可预测，从而允许使用更大的学习率：

1. BN 重新参数化使损失曲面更平滑
2. 梯度的幅度更稳定 (不会在某方向上极大或极小)
3. 可以使用更大的学习率 → 更快收敛
4. 梯度更可靠 → 训练更稳定

10.3 训练和推理的区别

方面 训练 推理

均值 / 方差来源 当前 batch 全局 running 统计量

每次结果 不同 (依赖 batch 组成) 确定性

梯度 有 (通过 batch 统计量) 无

注意：小 batch 时，batch 统计量不准确，可能导致性能下降。此时可考虑：

- Group Normalization：在通道维度分组标准化
- Layer Normalization：在特征维度标准化 (Transformer 默认)

- Instance Normalization: 在每个样本每个通道上标准化 (风格迁移)

```
import torch
import torch.nn as nn
```

不同归一化方法的适用场景

```
normdict = {
    'batch': nn.BatchNorm2d(64), # CNN, 大batch
    'layer': nn.LayerNorm(64), # Transformer, RNN
    'instance': nn.InstanceNorm2d(64), # 风格迁移
    'group': nn.GroupNorm(8, 64), # 小batch的CNN (8组)
}

- - -
```

十一、前沿方向

方向 说明 核心思想

神经架构搜索 (NAS) 自动化网络设计 用RL / 进化 / 梯度搜索最优架构

自监督学习 BERT、SimCLR等预训练方法 从数据自身构造监督信号

轻量化模型 MobileNet、EfficientNet等 减少参数量和计算量, 部署到移动端

图神经网络 (GNN) 处理非欧几里得数据 消息传递框架: 聚合邻居信息

Vision Transformer 将Transformer应用于视觉 图像分patch后作为序

多模态学习 文本 - 图像 - 语音的统一建模 CLIP、BLIP等跨模态对齐

扩散模型 逐步去噪生成 SDE / ODE框架, Sora / SD等

状态空间模型 Mamba、S4等 线性复杂度替代Transformer

- - -

十二、常见误区与最佳实践

常见误区

误区 正确理解

"网络越深越好" 没有残差连接时, 过深反而退化 (不是过拟合, 是训练困难)

" A d a m总是比 S G D好 " A d a m收敛快但可能泛化差, S G D + M o m e n t u m + 调度往往泛化更好

" B a t c h N o r m总是有帮助 " 小 b a t c h时 B N不稳定, 在线学习 / 强化学习场景不适合

" D r o p o u t越多越好 " 过高 D r o p o u t 率会降低模型容量, 需要调参

" G A N比 V A E生成质量高, 所以应该总用 G A N " G A N训练困难, V A E有更好的理论性质, 扩散模型

" 迁移学习就是微调最后一层 " 需要根据目标任务与源任务的相似度决定微调策略

" R e L U总是最好的激活函数 " T r a n s f o r m e r中 G E L U更好, 某些场景 S w i s h / L e a k y R e L U

最佳实践

- 1 . 从简单开始: 先跑通小模型和少量数据, 确认流程无误再扩大
- 2 . 使用预训练权重: 在视觉和 N L P任务中, 预训练 + 微调几乎总是优于从头训练
- 3 . 监控训练过程: 跟踪训练 / 验证损失、梯度范数、学习率、各层激活统计量
- 4 . 梯度检查: 实现自定义层 / 损失时, 用数值梯度验证自动微分的正确性
- 5 . 混合精度训练: 使用 F P 1 6 加速训练, 但注意梯度缩放防止下溢
- 6 . 实验可复现: 固定随机种子、记录超参数、版本控制代码

深度学习训练的标准模板

```
import torch
from torch.utils.data import DataLoader
from torch.cuda.amp import autocast, GradScaler

model = MyModel().cuda()
optimizer = torch.optim.AdamW(model.parameters())
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, num_epochs)
scaler = GradScaler() # 混合精度

for epoch in range(num_epochs):
    model.train()
    for batch in trainloader:
        x, y = batch[0].cuda(), batch[1].cuda()

        optimizer.zero_grad()
        with autocast(): # 混合精度前向传播
            pred = model(x)
            loss = criterion(pred, y)
```



```

scaler.scale(loss).backward() # 缩放梯度
scaler.unscale(optimizer)
torch.nn.utils.clip_grad_norm(model.parameters(), n
scaler.step(optimizer)
scaler.update()

scheduler.step()

# 验证
model.eval()
with torch.no_grad():
    val_loss = evaluate(model, val_loader)
    print(f"Epoch {epoch}: train_loss={loss.item():.4f}

```

核心总结：深度学习的知识体系从基础的FNN到CNN、RNN，再到Transformer和扩散模型，
 +
 优化训练"的范式。理解反向传播的数学原理、梯度消失 / 爆炸的根源、各种正则化技术的本质，是掌握所有进